序列拼接

seq | 排序与相邻交换 | C++17

1. 题意与记号

将 n 个有编号的非空数组按任意排列拼接。若拼接结果为 b，权值是各位置下标乘该位置数值之和。求最大值，或按排列方案计数的非严格次大值。数组内容相同仍按不同编号区分。

记第 i 个数组长度为 L_i，元素总和为 S_i，数组内部贡献为 C_i。若它前面已有 len 个元素，其贡献为 C_i + len × S_i。

```
C_i = sum(j * a[i][j], j=1..L_i)
w = sum(C_i) + sum(prefix_length(i) * S_i)
```

2. 相邻交换决定最优顺序

考虑相邻数组 X、Y，它们之前的总长度相同。除两者之间的相互贡献外，其他贡献均不变。X 放在 Y 前面，相比 Y 放在 X 前面多出的权值为：

```
w(X,Y) - w(Y,X) = L_X * S_Y - L_Y * S_X
```

因此按 S_i / L_i 非递减排序。实现时使用交叉乘法比较 S_X × L_Y 与 S_Y × L_X，避免浮点误差。

任意排列都可以通过交换相邻逆序对变成该顺序，每次交换都使权值不减，因此排好序的权值就是最大值 W。等比值的数组之间交换不改变权值。

3. 计算最大值

读入时累计所有 C_i。排序后从左到右维护 len，加入 len × S_i，然后令 len += L_i。数组内容在读完后无需保存。

非严格次大值

4. 结论

在一个最优顺序中，交换相邻数组 i、i+1 的损失记为 D_i。所有 D_i 非负。令 D 为其中最小值，则 k=2 的答案为 W-D。

```
D_i = L_i * S_(i+1) - L_(i+1) * S_i
D = min(D_i, i=1..n-1)
answer = W (k=1)
answer = W - D (k=2)
```

5. 存在等比值数组的情况

若最优顺序中有相邻数组比值相同，D=0。交换这两个不同编号的数组便得到另一个排列，且权值仍为 W。于是非严格次大值就是 W。

只要有两个数组的比值相同，排序后必有相邻的一对相同比值，所以该情况不会被遗漏，包括数组元素全为零的情况。

6. 所有比值不同的情况

此时最优顺序唯一，且所有 D_i>0。把任何非最优排列用相邻逆序交换排成最优顺序。所有交换的增益都为正。

观察最后一次交换：交换之后已经是最优顺序，因此交换的两个数组在最优顺序中也相邻。这次交换的增益恰是某个 D_i，至少为 D。整个排序过程的总增益至少为 D，所以任何非最优排列的权值都不超过 W-D。

另一方面，在最优顺序中交换损失等于 D 的相邻一对，确实得到权值 W-D 的不同排列。这个上界可以达到，因此它就是次大值。

实现与验证

7. 复杂度与整数范围

记全部数组长度之和为 M。读入和计算内部贡献用 O(M) 时间，排序 O(n log n)，扫描 O(n)。总时间 O(M+n log n)，额外空间 O(n)。

M<=10^6，元素不超过 10^6。权值至多 10^6 × M(M+1)/2，不超过约 5.000005×10^17，64 位有符号整数足够。长度、总和、贡献及交叉乘积均按 long long 计算。

对于两个不同数组，其长度之和至多 M，故交叉乘积 S_X × L_Y <=10^6 × L_X × L_Y <=2.5×10^17，同样不会溢出。

8. 实现要点

比较器只返回严格小于；比值相同不额外打破平局也没有问题。n>=2，因而一定存在相邻一对，可直接取最小交换损失。

代码默认使用题面规定的 seq.in / seq.out。若在标准输入输出环境运行，编译时加 -DLOCAL。

9. 样例核对

样例 1 最优编号顺序为 2、3、1，权值 26。相邻交换最小损失为 1，因此样例 2 的非严格次大值为 25。样例 3 有相同比值的数组，非严格次大值等于最大值 19。

10. 本地验证记录

三个题面样例均在标准输入输出与文件 IO 两种模式下通过。另用 400 组小数据枚举全部编号排列，分别核对最大值与非严格次大值，共 800 次检查。随机种子为 20261011。

最大规模检查覆盖 n=200000、M=1000000 及元素上界。本地编译使用 C++17 -O2；这些检查用于验证实现，不替代上述证明，也不是官方评测结论。